



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EICTA: Second Project Delivery - Neo4j & BPMN

Author(s): David Loredan Barb (10664390)

Letizia Castellani (11016807)

Federica Stefanizzi (10727743)

Rocco Tonon (10740668)

Isabella Zamboni (10929976)

Group Number: 1

Academic Year: 2023-2024

Contents

Contents	i
1 INTRODUCTION	1
2 DATABASE	3
3 DATABASE DETAILS	9
3.1 Nodes	9
3.1.1 User	9
3.1.2 ISA Hierarchy's Assumption:	9
3.1.3 Song & Podcast:	10
3.1.4 Category:	11
3.1.5 Playlist:	12
3.2 Relationships	13
3.2.1 LISTENS_TO:	13
3.2.2 IS_PART_OF:	14
3.2.3 AGGREGATE:	14
3.2.4 TAGS:	15
3.2.5 Relationship Cardinalities:	16
4 QUERIES	17
4.1 Creation/Delete/Update Queries	17
4.1.1 Creation Queries:	17
4.1.2 Deletion Queries:	18
4.1.3 Update Queries:	20
4.2 Complex Queries	21
4.2.1 3 queries with at least 2 nodes in the MATCH statement and conditions:	21

4.2.2	2 queries with at least 2 nodes in the MATCH statement, conditions and aggregation without a WITH statement	23
4.2.3	2 queries with at least 2 nodes in the MATCH statement, conditions and a WITH statement	24
4.2.4	2 queries with at least 3 nodes in the MATCH statement, conditions and multiple WITH statements	26
4.2.5	1 query with the shortestPath(...) function	28
5	BPMN	29
5.1	BPMN DIAGRAM:	29
6	CONCLUSION	31

1 | INTRODUCTION

Entering the second phase of EICTA's project, the database management system takes center stage, with Neo4j chosen over MongoDB. This shift stems from a recognition that the application's essence lies in understanding and leveraging the intricate relationships between users and their music preferences in the 'musicApp' system.

The query language in Neo4j, Cypher, is tailored for traversing and querying graph data, making it more expressive and natural for operations involving relationships. This eased the development process and allowed for more straightforward and efficient retrieval of interconnected data compared to MongoDB's query language.

Scalability was another pivotal factor. While both databases offer scalability options, Neo4j's architecture is specifically designed for handling graph data and traversing relationships efficiently, making it a more natural fit for applications where relationships are a primary focus. MongoDB, although scalable, might not perform as optimally when relationships become increasingly complex and numerous.

So, the deliberate choice of Neo4j prioritizes relationships over mere data storage, enabling the exploration of connections through shared playlists, collaborative tagging, and collective listening experiences.

Overall, these are the reasons of our choice for the type of NoSQL DataBase.

2 | DATABASE

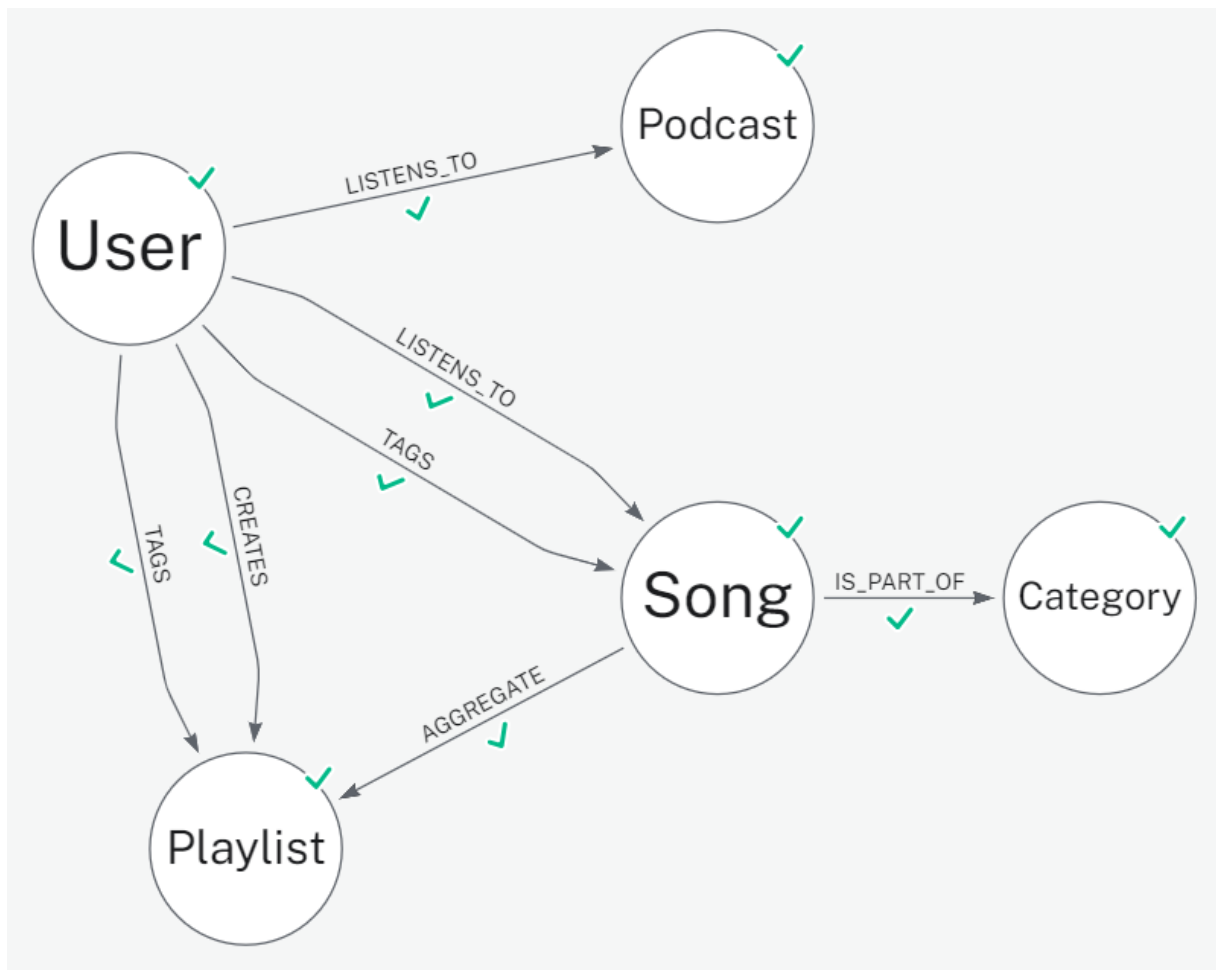


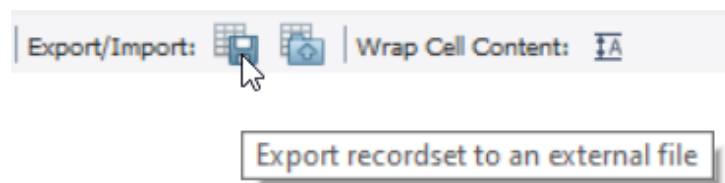
Figure 2.1: Neo4j's Schema

The schematic representation illustrated above took inspiration mainly from the ER model created and visualized in the first delivery of the project. However, there are slight differences since the entities are represented as nodes and relationships as arcs (directed edges). Cardinalities aren't represented here since Neo4j doesn't require them.

The database is created by exporting all the .csv files from each table of 'MusicApp' MySQL DataBase. For simplicity, only 'Category' table is queried and then exported. Here's how to do it:



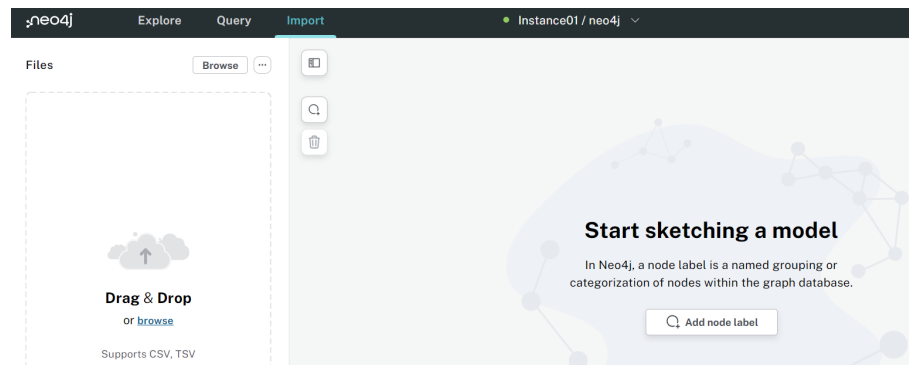
Then, the export phase:



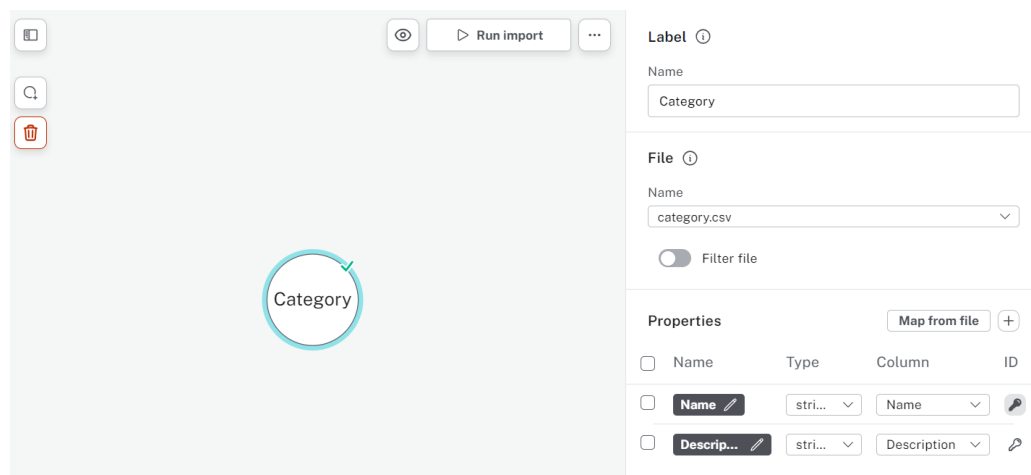
All the tables in .csv format should be in the same folder, like this:

PROJECT_EICTA > musicApp_data_from_MySQL				
Name	Date modified	Type	Size	
category.csv	12/1/2023 3:19 PM	Microsoft Excel Co...	5 KB	
playlist.csv	12/1/2023 3:45 PM	Microsoft Excel Co...	3 KB	
podcast.csv	12/1/2023 3:45 PM	Microsoft Excel Co...	3 KB	
song.csv	12/1/2023 3:46 PM	Microsoft Excel Co...	7 KB	
song_playlist.csv	12/1/2023 3:47 PM	Microsoft Excel Co...	13 KB	
user.csv	12/1/2023 4:00 PM	Microsoft Excel Co...	4 KB	
user_playlist.csv	12/1/2023 3:47 PM	Microsoft Excel Co...	21 KB	
user_song.csv	12/1/2023 3:47 PM	Microsoft Excel Co...	21 KB	
userlistenstopodcast.csv	12/1/2023 3:54 PM	Microsoft Excel Co...	26 KB	
userlistenstosong.csv	12/1/2023 3:53 PM	Microsoft Excel Co...	27 KB	

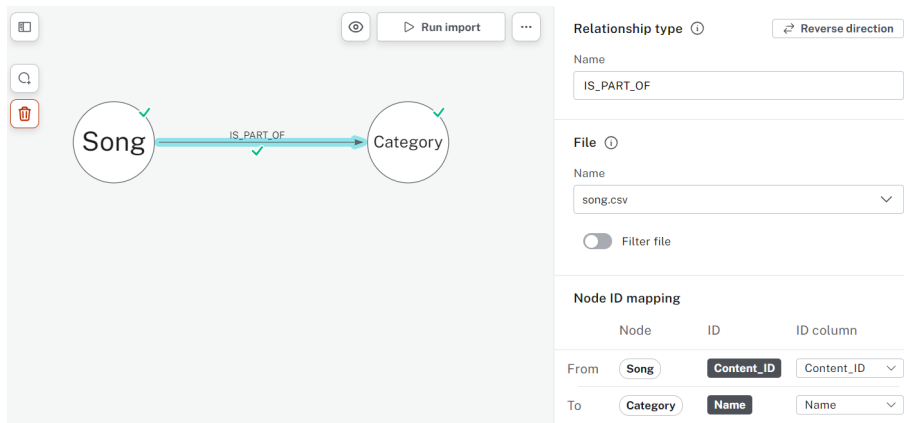
Then, in 'Import' section of Neo4j AuraDB instance, 'Drag&Drop' the folder of all .csv tables:



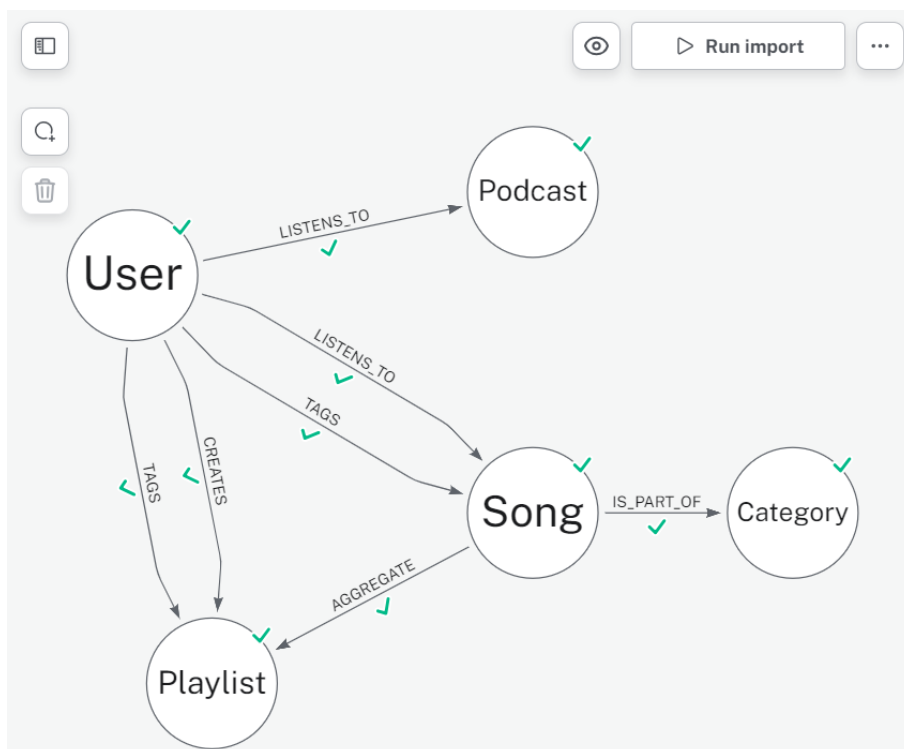
To create a node, 'add node label' to start sketching a model, naming it and later selecting the respective .csv file to perform the import. Finally, 'Map from file' all the attributes and assign them the right data type (unfortunately there isn't the selection of only Date or only Time data types, it is required later to change the types). A unique ID can be assigned based on the primary attribute by clicking the key symbol on the right, like in the image below:



Relationships can be created by clicking on the border of the starting node and hovering the mouse to the another node of the relationship. After this, select the .csv file of the bridge table if there is one, if not choose the file of the starting node. Finally, 'Node ID mapping' by selecting the primary keys of both nodes, like this:



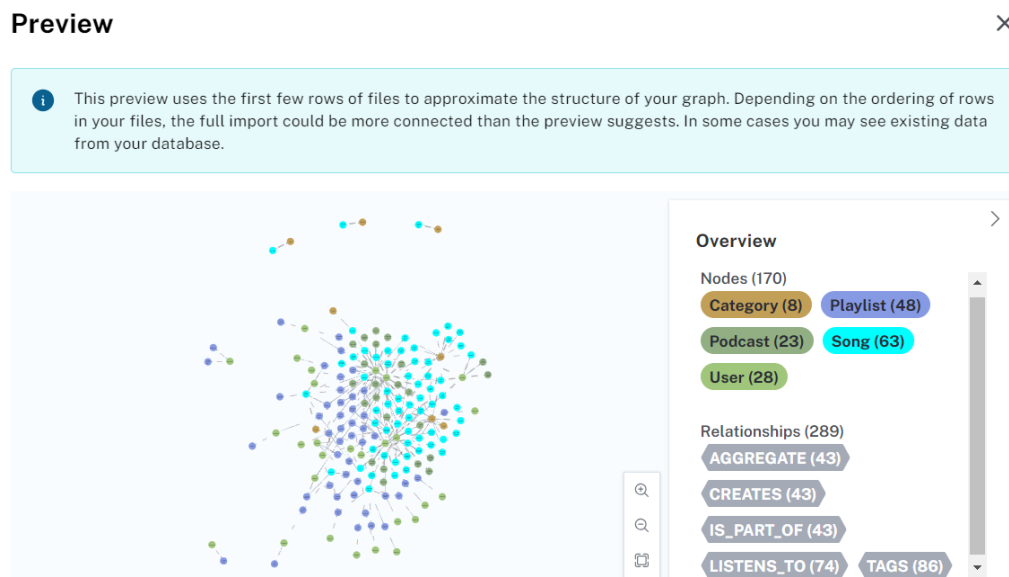
Finally, the entire database is created by following the ER model and by clicking on "Run Import". The Neo4j DB can be successively downloaded in a .zip format in order to be able to upload it in a single click without performing all these long operations of creating node labels.



One last thing before starting working with the DB through queries is that in Neo4j's 'Import' section the node label creation doesn't have DATE and TIME data types, only DATETIME, which isn't good for attributes like dates of subscriptions or duration of songs. To resolve that, a query needs to be performed in order to change all these data types into the correct form. This is the query:

```
1 MATCH (p:Playlist)
2 SET p.CreationDate = date(p.CreationDate);
3 MATCH (usr:User)
4 SET usr.RegistrationDate = date(usr.RegistrationDate);
5 MATCH (s:Song)
6 SET s.PublicationDate = date(s.PublicationDate);
7 MATCH (p:Podcast)
8 SET p.PublicationDate = date(p.PublicationDate);
9 MATCH (p:Podcast)
10 SET p.Duration = time(p.Duration);
11 MATCH (s:Song)
12 SET s.Duration = time(s.Duration);
```

This is the preview of the DB created:



More details about the created nodes, relationships and assumptions are explained in the next chapter "DATABASE DETAILS".

3 | DATABASE DETAILS

3.1. Nodes

3.1.1. User

Label ⓘ

Name

User

File ⓘ

Name

user.csv

☐ Filter file

Properties Map from file +

☐	Name	Type	Column	ID
☐	Email	string	Email	
☐	Username	string	Username	
☐	Password	string	Password	
☐	RegistrationDa...	string	RegistrationDate	
☐	IsPremium	boolean	IsPremium	

The first identified node is the one representing the **User** who is using the music application. Users are recognised by an Email, a RegistrationDate, a Username, a Password, and an attribute called IsPremium, which is assumed to be a boolean variable indicating whether or not a given user pays a monthly fee.

3.1.2. ISA Hierarchy's Assumption:

ISA hierarchy of **Audio Content** (as it was done in the first delivery of the project with MySQL) has not been implemented for the simple reason that the children are Total and Exclusive, which means that a father node shouldn't be created. Therefore, Podcast and Song entities are created as single nodes. These nodes share some attributes (the ones previously attributed to Audio_Content). More in detail, the common attributes are a unique Content_ID, a Title, a Duration, and a PublicationDate. Nevertheless, those two nodes present also their individual attributes.

3.1.3. Song & Podcast:

Label ⓘ

Name

Song

File ⓘ

Name

song.csv

☐ Filter file

Properties Map from file +

☐	Name	Type	Column	ID
☐	Content_ID ⓘ	integer	Content_ID	☒
☐	Title ⓘ	string	Title	☐
☐	Duration ⓘ	string	Duration	☐
☐	PublicationD... ⓘ	string	PublicationDate	☐
☐	Artist ⓘ	string	Artist	☐
☐	Text ⓘ	string	Text	☐

Label ⓘ

Name

Podcast

File ⓘ

Name

podcast.csv

☐ Filter file

Properties Map from file +

☐	Name	Type	Column	ID
☐	Content_ID ⓘ	integer	Content_ID	☒
☐	Title ⓘ	string	Title	☐
☐	Duration ⓘ	string	Duration	☐
☐	Publicatio... ⓘ	string	Publication...	☐
☐	Author ⓘ	string	Author	☐

Song is further identified by Text and Artist, whilst Podcast includes a column disclosing the authors. As also assumed in the first part of the project, it is critical to note that, due to design choices and assumptions, the Artist who wrote a Song instance and the Author of a Podcast are represented as attributes of their respective nodes, rather than as nodes themselves. Finally, a significant assumption is that a Song may have been produced by more than one Artist and a Podcast by more than one Author: if this is the case, the names of those who participated in the song’s writing or the podcast production will be presented in a single node in a unique string.

3.1.4. Category:

Label ⓘ

Name

Category

File ⓘ

Name

category.csv

Filter file

Properties

Map from file +

☐	Name	Type	Column	ID
☐	Name	string	Name	
☐	Description	string	Description	

Another node identified in the text is Category, which is assumed to be defined by a unique Name, and a Description. For a more intuitive understanding, each Song is presumed to belong to exactly one category.

3.1.5. Playlist:

Label ⓘ

Name

File ⓘ

Name

☐ Filter file

Properties
Map from file +

☐	Name	Type	Column	ID
☐	Playlist_ID ✎	integer ▾	Playlist_ID ▾	
☐	Title ✎	string ▾	Title ▾	
☐	CreationDate ✎	string ▾	CreationDate ▾	
☐	IsPublic ✎	boolean ▾	IsPublic ▾	
☐	Email ✎	string ▾	Email ▾	

The last identified node is **Playlist**, which is recognised by a distinctive `Playlist_ID`, a `Title`, a `CreationDate`, and an attribute called `IsPublic`, which is presumed to be a boolean variable indicating whether or not a playlist may be accessed by everyone.

3.2. Relationships

3.2.1. LISTENS_TO:

Relationship type ⓘ

↔ Reverse direction

Name

LISTENS_TO

File ⓘ

Name

userlistenstosong.csv

▼

☐ Filter file

Node ID mapping

	Node	ID	ID column
From	User	Email	Email ▼
To	Song	Content_ID	Content_ID ▼

Need help?

Properties

Map from file +

☐	Name	Type	Column
☐	When	datetime ▼	When ▼
☐	Whole	boolean ▼	Whole ▼

Relationship type ⓘ

↔ Reverse direction

Name

LISTENS_TO

File ⓘ

Name

userlistenstopodcast.csv

▼

☐ Filter file

Node ID mapping

	Node	ID	ID column
From	User	Email	Email ▼
To	Podcast	Content_ID	Content_ID ▼

Need help?

Properties

Map from file +

☐	Name	Type	Column
☐	When	datetime ▼	When ▼
☐	Whole	boolean ▼	Whole ▼

The first identified relation is **LISTENS_TO**, which links User with Podcast, and User with Song. This relation is composed of two attributes called When and Whole, which respectively indicate when a User listened to a Song or a Podcast and whether it was listened to in its entirety. A User may listen to an unlimited amount of Podcast or Song, and these last may be listened to by an unlimited number of users.

3.2.2. IS_PART_OF:

Relationship type ⓘ

↔ Reverse direction

Name

IS_PART_OF

File ⓘ

Name

song.csv

☐ Filter file

Node ID mapping

	Node	ID	ID column
From	Song	Content_ID	Content_ID
To	Category	Name	Name

Song and Category are connected by the relation IS_PART_OF that defines whether a Song is part of a particular Category. While a Category can contain indefinite instances of Song, a song is assumed and required to be part of exactly one category.

3.2.3. AGGREGATE:

Relationship type ⓘ

↔ Reverse direction

Name

AGGREGATE

File ⓘ

Name

song_playlist.csv

☐ Filter file

Node ID mapping

	Node	ID	ID column
From	Song	Content_ID	Content_ID
To	Playlist	Playlist_ID	Playlist_ID

Need help?

Properties

Map from file +

☐	Name	Type	Column
☐	InsertionDate	datetime	InsertionDate

Song and Playlist are connected via the relation AGGREGATE that is solidified by the InsertionDate attribute, which provides the precise day on which a Song was inserted into a Playlist. Realistically, in order for a playlist to be qualified as such, it must contain at least one song and, while a Playlist is assumed to be created by exactly one User, each User is allowed to create an indefinite number of playlists.

A peculiar assumption of this database aims to be sure that, in order to prevent duplicates, a Song can be aggregated into a Playlist only once.

3.2.4. TAGS:

Relationship type ⓘ

Reverse direction

Name

TAGS

File ⓘ

Name

user_song.csv

Filter file

Node ID mapping

	Node	ID	ID column
From	User	Email	Email
To	Song	Content_ID	Content_ID

Need help?

Properties

Map from file +

☐	Name	Type	Column
☐	TagDate	datetime	TagDate

Relationship type ⓘ

Reverse direction

Name

TAGS

File ⓘ

Name

user_playlist.csv

Filter file

Node ID mapping

	Node	ID	ID column
From	User	Email	Email
To	Playlist	Playlist_ID	Playlist_ID

Need help?

Properties

Map from file +

☐	Name	Type	Column
☐	TagDate	datetime	TagDate

Additionally, the Music Application is required to store whenever a Song or Playlist is tagged as favourite by a User, and this is the rationale behind the creation of the two relations **TAGS**, as they both include the attribute TagDate indicating when a Playlist or a Song has been marked as favourite. During the establishment process of these relations, several assumptions have been taken care of. While each User can tag indefinite Song or Playlist instances as favourites, and those instances can be tagged as favourites by an indefinite number of users, a user is actually unable to mark the same Song or Playlist as a favourite more than once, even if tagged in different days; nevertheless, different songs and playlists can be marked as favourites on the same day.

3.2.5. Relationship Cardinalities:

In Neo4j, **cardinalities** are not a primary concern, unlike in the Entity-Relationship (ER) model. The relationship is characterised by an explicit direction, an arc in short, while in the ER model relationships are inherently bidirectional. This is done because in Neo4j it's essential to explicitly specify their directions.

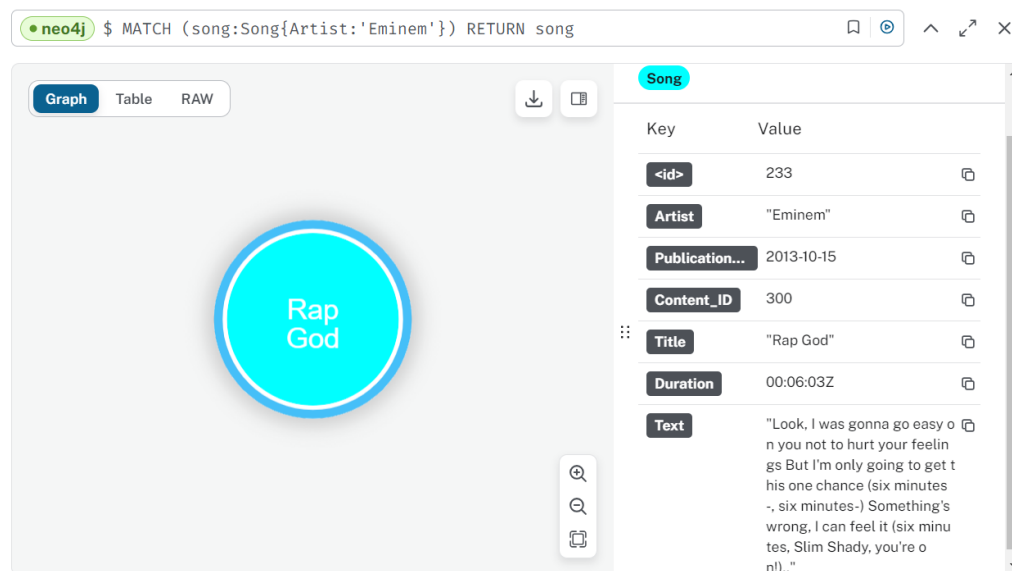
4 | QUERIES

4.1. Creation/Delete/Update Queries

4.1.1. Creation Queries:

- 1ST CREATION QUERY:

```
1 CREATE (song: Song {Artist: 'Eminem', PublicationDate: DATE('2013-10-15'),
  Content_ID:300, Title: 'Rap God', Duration: TIME('00:06:03'), Text: 'Look, I was
  gonna go easy on you not to hurt your feelings But I\'m only going to get this
  one chance (six minutes-, six minutes-) Something\'s wrong, I can feel it (six
  minutes, Slim Shady, you\'re on!)..'})
2
3
```



The image shows the Neo4j web interface. At the top, a Cypher query is entered: `$ MATCH (song:Song{Artist:'Eminem'}) RETURN song`. Below the query, the results are displayed in a table view. The table has two columns: 'Key' and 'Value'. The 'Key' column lists the properties of the 'Song' node: <id>, Artist, Publication..., Content_ID, Title, Duration, and Text. The 'Value' column shows the corresponding values: 233, "Eminem", 2013-10-15, 300, "Rap God", 00:06:03Z, and "Look, I was gonna go easy o n you not to hurt your feelin gs But I'm only going to get t his one chance (six minutes -, six minutes-) Something's wrong. I can feel it (six minu tes, Slim Shady, you're o n!)..".

Key	Value
<id>	233
Artist	"Eminem"
Publication...	2013-10-15
Content_ID	300
Title	"Rap God"
Duration	00:06:03Z
Text	"Look, I was gonna go easy o n you not to hurt your feelin gs But I'm only going to get t his one chance (six minutes -, six minutes-) Something's wrong. I can feel it (six minu tes, Slim Shady, you're o n!).."

This query is creating a new node in the graph. This node is labelled as “Song” with several properties. The properties specified within the curly braces are: Artist, PublicationDate, Content_ID, Title, Duration, and Test. This Cypher command does not specify any relationships to other nodes.

- 2ND CREATION QUERY:

```
1 CREATE (user: User{Email: 'penguinynm3@outlook.com', RegistrationDate: DATE('2023-10-15'), Username: 'plmnpenguin0', IsPremium: true, Password: 'f94ngFGon4VBC_' })
```

neo4j \$ MATCH (usr:User{Username:'plmnpenguin0'}) RETURN usr

Graph Table RAW

Node details

User

Key	Value
<id>	232
Email	"penguinynm3@outlook.com"
RegistrationDate	2023-10-15
Username	"plmnpenguin0"
IsPremium	true
Password	"f94ngFGon4VBC_"

Started streaming 1 record after 44ms and completed after 46ms.

This query is creating a new node in the graph. The CREATE (user:User) statement is creating a new node with the label "User." The properties of this node are enclosed within the curly braces. More in detail the properties specified within the curly braces are: Email, RegistrationDate, Username, IsPremium (that is a Boolean value) and Password. This query does not include any relationship.

4.1.2. Deletion Queries:

- 1ST DELETION QUERY:

```
MATCH (usr:User{Email:"igerrietsb@meetup.com"})-[r]-()
DETACH DELETE usr,r
```

This query is used to delete a user node from the graph database, including all its relationships, regardless of the other nodes they connect to. The MATCH clause identifies the node to be deleted. It looks for a node with the label User that has an Email property with the value "igerriets@meetup.com".

The following statement: `"-[r]-()"` matches all the relationships `r` that the identified user node `usr` has, without specifying the type of relationships or the other nodes they are connected to.

The `"DETACH DELETE usr, r"` is the command that removes both the node and all of its connections. Before deleting the node itself, use `DETACH DELETE` to automatically erase any relationships connected to the node user. By doing this, it is made sure that the database is free of orphaned relationships, which would compromise the graph's integrity.

This query will result in the complete removal of the user with the supplied email address from the database, along with all relationships that were linked to this user node. When a node needs to be removed from the graph along with all of its connections, this is a typical procedure.

- 2ND DELETION QUERY:

```
MATCH (cat:Category{Name: 'Alternative'})-[r]-()  
DETACH DELETE cat, r
```

This query is used to find and delete a specific category node from the Neo4j graph database, along with all its relationships, regardless of what other nodes those relationships connect to.

The match statement: `"MATCH (cat:Category Name: 'Alternative')"` is used to match the node that has a label `Category` and a property `Name` with the value `Alternative`.

The following statement: `"-[r]-()"` matches any relationships `r` that the category node `cat` has with any other node. The empty parentheses `()` indicate that it does not matter what other node it is connected to; any relationships will be matched.

The last pattern, "DETACH DELETE cat, r" deletes all of the relationships associated with the node cat as well as the node itself. Neo4j does not allow to delete a node without also deleting its relationships, hence this is required. Consequently, this query's ultimate result is to eliminate any associations associated with the "Alternative" category from the database, thereby clearing up any links that might have been left behind.

4.1.3. Update Queries:

```
1 MATCH (usr:User{Email:"igerrietsb@meetup.com"})
2 SET usr.IsPremium = false
3 SET usr.Username = 'gerrietbane1234'
4 RETURN usr
```

The node before updating it:

neo4j \$ MATCH (usr:User{Email:"igerrietsb@meetup.com"}) RETURN usr

Graph Table RAW

mhenni-ngtonb

Node details

User

Key	Value
<id>	49
Email	"igerrietsb@meetup.com"
Registration...	2023-04-28
Username	"mhenni-ngtonb"
IsPremium	true
Password	"uT0,i+7CE/zD#bylj"

Started streaming 1 record after 14ms and completed after 15ms.

The same node, after updating it:

Graph Table RAW

gerrietbane1234

Node details

User

Key	Value
<id>	49
Email	"igerrietsb@meetup.com"
Registration...	2023-04-28
Username	"gerrietbane1234"
IsPremium	false
Password	"uT0,i+7CE/zD#bylj"

Set 2 properties

Started streaming 1 record after 45ms and completed after 47ms.

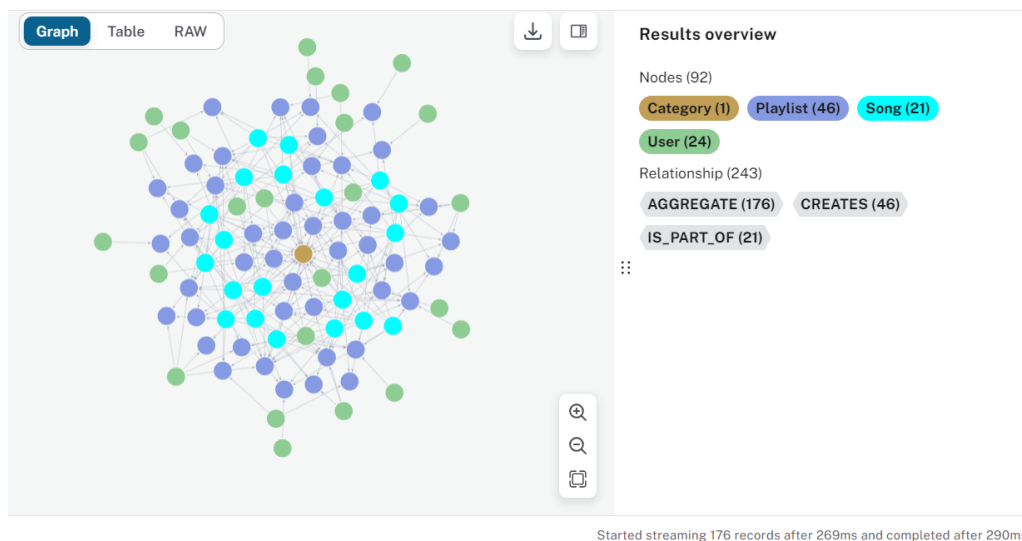
The purpose of this query is to update some values of the User that is associated with the following email: 'igerrietsb@meetup.com'. More in detail, this query changes the value of the attribute IsPremium from True to False and it changes the Username of the node to 'gerrietbane123'.

4.2. Complex Queries

4.2.1. 3 queries with at least 2 nodes in the MATCH statement and conditions:

- 1ST QUERY:

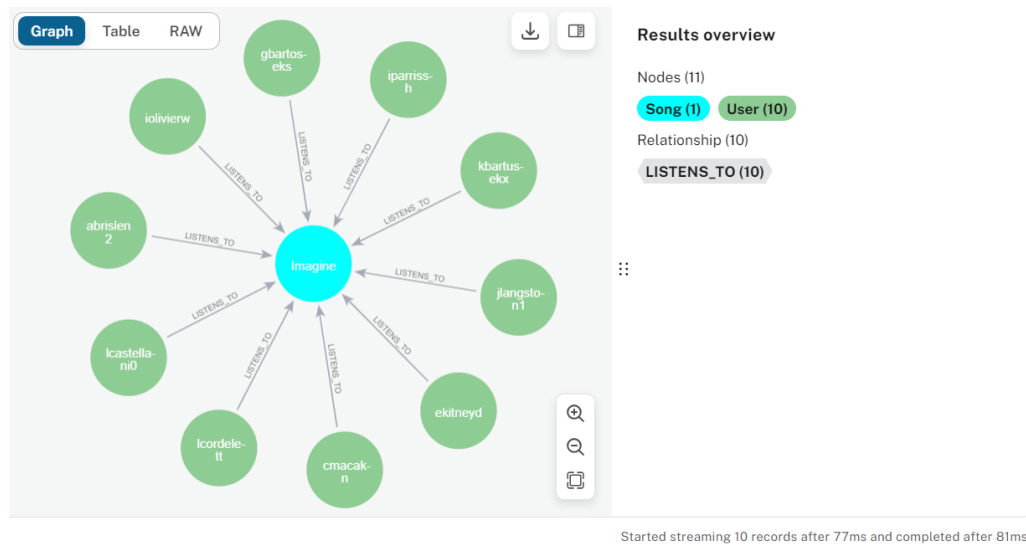
```
1 MATCH (user:User)-[r1:CREATES]→(playlist:Playlist)←[r2]-(song:Song)-[r3]→
  (category:Category {Name: 'Rock'})
2 RETURN user, r1, playlist, r2, song, r3, category
```



This query seeks the users who have created playlists, where each playlist contains at least one song that is associated with the 'Rock' category. The condition statement is represented by the curly brackets.

- 2ND QUERY:

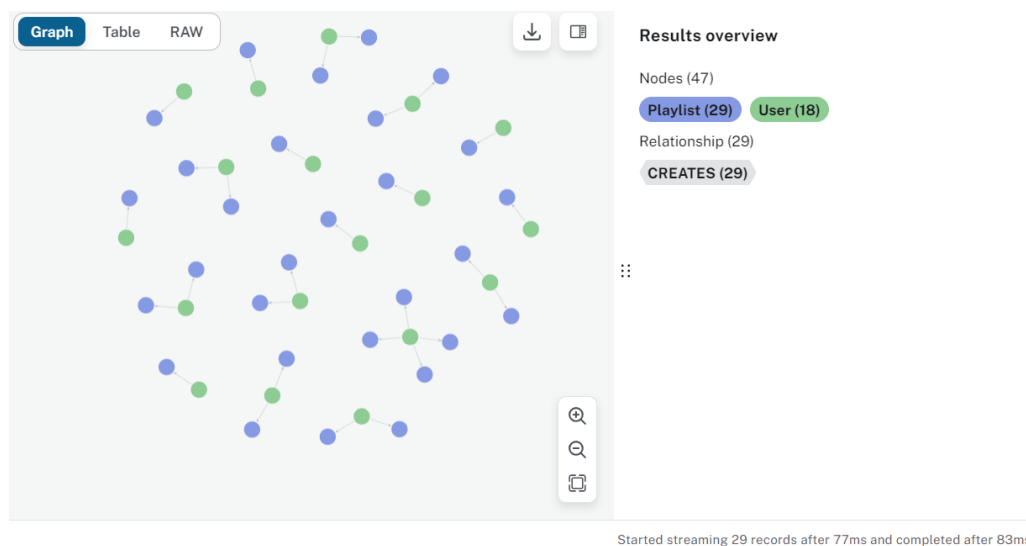
```
1 MATCH (usr:User)-[r1:LISTENS_TO]→(song:Song)
2 WHERE song.Title = 'Imagine'
3 RETURN usr, r1, song
```



This query searches for the users who listen to the song titled 'Imagine'.

- 3RD QUERY:

```
1 MATCH (user:User)-[r1:CREATES]→(playlist:Playlist {IsPublic: true})
2 RETURN user, r1, playlist
```



This query searches for the users who create playlists that have the property of being 'public'.

4.2.2. 2 queries with at least 2 nodes in the MATCH statement, conditions and aggregation without a WITH statement

- 1ST QUERY:

```
1 MATCH (usr:User)-[:CREATES]→(playlist:Playlist)-[:AGGREGATE]-(song:Song)-[:IS_PART_OF]→
  (category:Category)
2 WHERE (category.Name = 'Rock' OR category.Name = 'Jazz')
3 RETURN usr.Username, COUNT(DISTINCT playlist) AS playlistCount
```

Table RAW

	usr.Username	playlistCount
1	"rcabbellq"	1
2	"abrislen2"	5
3	"ekitneyd"	1
4	"walsteadp"	1
5	"tgowland3"	1
6	"iolivierw"	1
7	"lcrewe5"	1
8	"vconnew1"	6
9	"wwhitnallb"	1

This query returns each user's username along with the number of unique playlists they have created that contain at least one song categorized as either 'Rock' or 'Jazz'.

- 2ND QUERY:

```
1 MATCH (u:User)-[:TAGS]→(s:Song)-[:IS_PART_OF]→(c:Category)
2 WHERE c.Name = 'Rock'
3 RETURN u.Username AS Username, COUNT(DISTINCT s) AS NumberOfTaggedSongs
```

	Username	NumberOfTaggedSongs
1	"lmalamorev"	6
2	"mdurrett13"	5
3	"gparkissonr"	6
4	"rnowell4"	5
5	"fromanc"	4
6	"tpresseyk"	6
7	"lcrewe5"	3
8	"rgustusu"	3
9	"vburehill0"	6

The overall purpose of this query is to find all users who have tagged songs, count the total number of 'Rock' songs each user has tagged, and return that count along with the Username of each user that respects the condition. "COUNT(DISTINCT s)" avoids counting same songs that have been tagged more than once by the same user.

4.2.3. 2 queries with at least 2 nodes in the MATCH statement, conditions and a WITH statement

- 1ST QUERY:

```

1 MATCH (u:User)-[:LISTENS_TO]-(s:Song), (u)-[:LISTENS_TO]-(p:Podcast)
2 WHERE s.Title = 'Hallelujah (Cover)'
3 WITH u, p
4 RETURN u.Username AS UserName, collect(p.Title) AS Podcasts

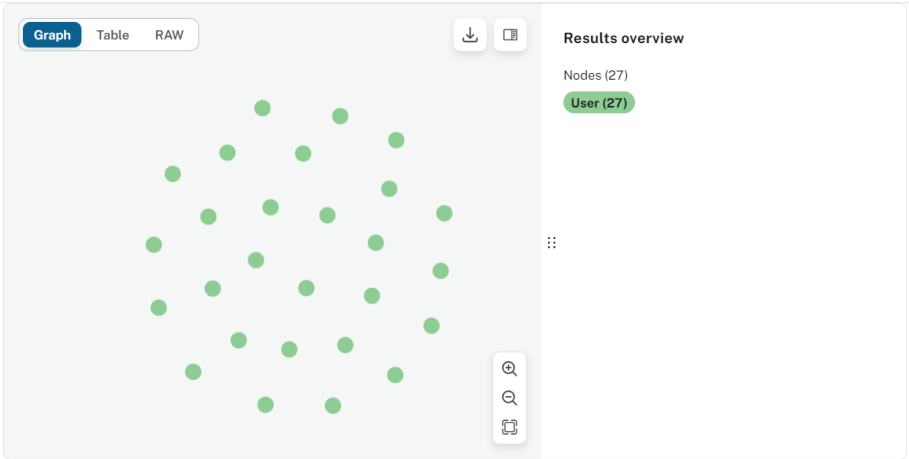
```

	UserName	Podcasts
1	"wwhitnallb"	["Burden of Guilt", "Huberman Lab", "MrBallen's Medical Mysteries", "Fresh Air", "Symptomatic: A Medical Mystery Podcast", "Crime Junkie", "48 Hours"]
2	"cmarneri"	["The Ben Shapiro Show", "The Mel Robbins Podcast", "Crime Junkie", "New Heights with Jason and Travis Kelce", "Stuff You Should Know", "48 Hours"]
3	"rcabbellq"	["Dateline NBC", "Burden of Guilt", "Conan O'Brien Needs A Friend", "MrBallen's Medical Mysteries", "20/20", "Very Scary People", "The Mel Robbins Podcast", "Fresh Air", "Heavyweight", "JoJo Siwa Now"]
4	"kbartusekx"	["The Ben Shapiro Show", "The Daily", "Morrison Mysteries", "48 Hours", "Murder in Apartment 12", "This American Life"]
5	"vburehill0"	["Hard Feelings with Jennette McCurdy", "The Daily", "Up First", "Stuff You Should Know", "48 Hours", "The Ben Shapiro Show"]

This query finds users who have listened to the song 'Hallelujah (Cover)' and also listened to podcasts. It then returns the username of each user (UserName) and a collection of podcast titles (Podcasts) that they have listened to.

- 2ND QUERY:

```
1 MATCH (u:User)-[r1:CREATES]→(p:Playlist)←[r2:AGGREGATE]-(s:Song)
2 WITH u, p, COUNT(s) AS songsCount
3 WHERE songsCount > 3
4 RETURN u AS User, r1, r2, COUNT(p) AS PlaylistsCount
```



	User	PlaylistsCount
1	(:User {Email: "tlummusf@rambler.ru", RegistrationDate: 2023-02-14, Username: "aboylanf", IsPremium: false, Password: "iI3_KB+tFpx/@dw\$z6"})	2
2	(:User {Email: "selias3@nationalgeographic.com", RegistrationDate: 2023-04-25, Username: "tnorfolk3", IsPremium: false, Password: "hU1<ki*EK%).45J2RPmA\$"})	1
3	(:User {Email: "gpridgeonq@fda.gov", RegistrationDate: 2023-09-01, Username: "rcabbellq", IsPremium: true, Password: "wI8,e>qH0oM8KcjmuHv= 9ux"})	1
4	(:User {Email: "ezimek9@histats.com", RegistrationDate: 2022-12-22, Username: "imulholland9", IsPremium: true, Password: "gI5!G1zeV4s@euNYu#D4y8Hl"})	2
5	(:User {Email: "psazio1@google.ca", RegistrationDate: 2023-09-11, Username: "vconnew1", IsPremium: false, Password: ...})	6

This query finds users who have created playlists, where each playlist aggregates multiple songs. It then counts the number of songs in each playlist, filters for playlists with more than 3 songs and returns information about the user, and the count of qualifying playlists (PlaylistsCount).

4.2.4. 2 queries with at least 3 nodes in the MATCH statement, conditions and multiple WITH statements

- 1ST QUERY:

```

1 MATCH (usr:User)-[:CREATES]-(playlist:Playlist)
2 WHERE playlist.CreationDate ≥ date('2023-01-01')
3 WITH usr, playlist
4 MATCH (playlist)-[]-(song:Song)
5 WITH usr, playlist, COLLECT(song) AS CollectionOfSongs
6 WITH usr, playlist, [elem IN CollectionOfSongs WHERE elem.PublicationDate ≤
  date('1990-01-01') | elem.Title] AS filteredCollectionOfSongs
7 RETURN usr.Username, playlist.Title, filteredCollectionOfSongs

```

usr.Username	playlist.Title	filteredCollectionOfSongs
1 "aboylanf"	"Meditation Mix"	["Hotel California", "Don't Stop Believin'", "Can't Help Falling in Love", "Livin' on a Prayer", "Stairway to Heaven", "Girls Just Want to Have Fun"]
2 "tnorfolk3"	"Chill Vibes"	["Hey Jude", "Purple Rain", "Thriller", "Let It Be"]
3 "rcabbellq"	"Country Roads"	["Hotel California", "Dancing Queen", "Like a Rolling Stone", "Don't Stop Me Now"]
4 "imulholland9"	"Hip-Hop Legends"	["Hey Jude", "Superstition", "What a Wonderful World", "Don't Stop Me Now"]
5 "vconnew1"	"Electro Swing"	["Rocket Man", "What a Wonderful World", "Here Comes the Sun", "All You Need Is Love"]

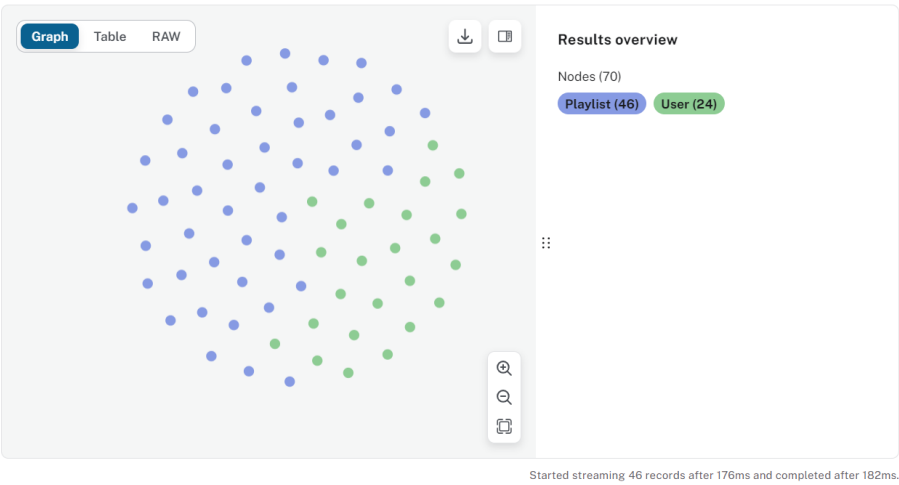
This query returns the usernames of users who have created a playlist in 2023 and for each playlist put all songs made before the 90s into a 'filtered' collection.

- 2ND QUERY:

```

1 MATCH (u:User)-[:CREATES]→(p:Playlist)
2 WITH u, p
3 MATCH (p)←[:AGGREGATE]-(s:Song)-[:IS_PART_OF]→(c:Category {Name: 'Rock'})
4 WITH u, p, COLLECT(s) AS songs
5 WHERE p.CreationDate ≥ date('2022-12-01')
6 WITH u, p, SIZE(songs) AS SongsCount
7 RETURN u AS User, p AS Playlist, SongsCount

```

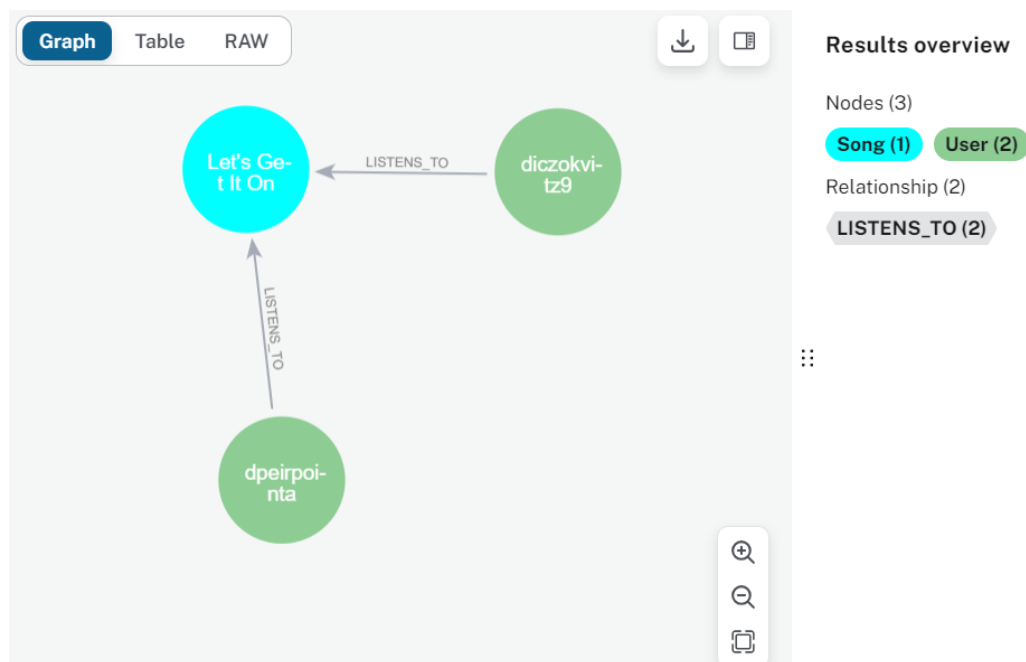


	User	Playlist	SongsCount
¹	<code>(:User {Email: "gpridgeonq@fda.gov", RegistrationDate: 2023-09-01, Username: "rcabbellq", IsPremium: true, Password: "wI8,e>qH0oM8KcjmUhV= 9ux"})</code>	<code>(:Playlist {Playlist_ID: 12784, CreationDate: 2023- 10-05, Email: "gpridgeonq@fda.gov", IsPublic: true, Title: "Country Roads"})</code>	3
²	<code>(:User {Email: "rmarriot2@amazon.com", RegistrationDate: 2023-04-14, Username: "abrislen2", IsPremium: false, Password: "sQ1}1o3y{*#h1sul"})</code>	<code>(:Playlist {Playlist_ID: 24286, CreationDate: 2023- 11-02, Email: "rmarriot2@amazon.com", IsPublic: false, Title: "R&B Soulmates"})</code>	4
³	<code>(:User {Email: "jlongstaffd@gizmodo.com", RegistrationDate: 2023-06-10, Username: "ekitneyd", IsPremium: true,</code>	<code>(:Playlist {Playlist_ID: 31491, CreationDate: 2023- 05-14, Email: "jlongstaffd@gizmodo.com",</code>	6

This query finds playlists created by a user, where each playlist aggregates songs that are part of the 'Rock' category. It then filters for playlists created on or after the first of December, 2022, calculates the count of songs for each playlist, and returns the user, playlist, and the count of songs for qualifying playlists.

4.2.5. 1 query with the shortestPath(...) function

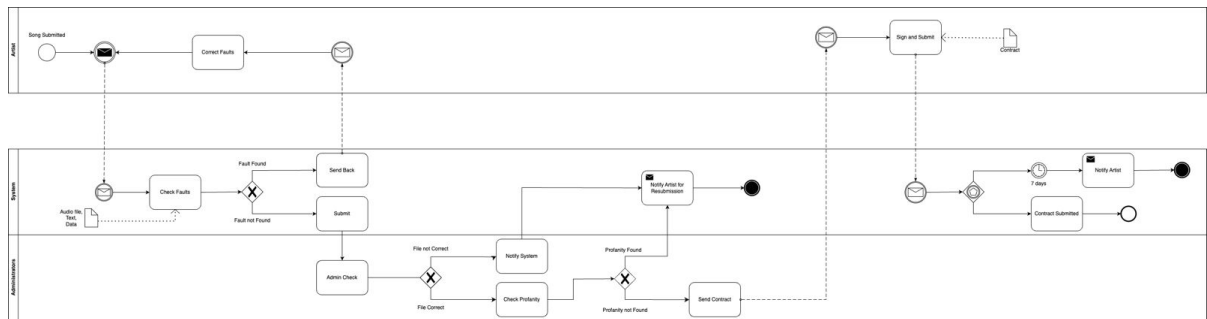
```
1 MATCH (usr1:User{Email: "iwatkin9@bbc.co.uk"}),
2     (usr2:User{Email: "ecayeta@stanford.edu"}),
3     path = ShortestPath((usr1)-[*]-(usr2))
4 RETURN path
```



This query is looking for the shortest path in the graph that connects the user with the email "iwatkin9@bbc.co.uk" to the user with the email "ecayeta@stanford.edu", and it returns this path. This could be used, for example, to find the closest connection between two people in a social network or a professional network graph.

5 | BPMN

5.1. BPMN DIAGRAM:



The provided BPMN diagram illustrates the intricate workflow for an artist's song submission process, involving three primary participants: the Artist, the System, and the Administrators.

The Artist initiates the process by submitting a song through a "Song Submitted" start event, employing an intermediate throwing message event to transmit the audio file, song lyrics, and related data as a single composite submission.

Upon receiving the submission, the System conducts an initial evaluation, utilizing an exclusive gateway to identify faults in the music file. If issues are detected, an iterative feedback loop ensues, prompting the Artist to rectify the faults and resubmit the song.

Once the System approves the submission, it proceeds to the Administrators' lane for further checks. This involves two sequential evaluations: the file correctness check and the profanity check. In case of inaccuracies or profanity, notifications are issued to the Artist, followed by process termination.

The diagram makes it clear that the term "termination" implies a complete halt of the process, necessitating a fresh submission from the Artist to restart. The notification to the Artist is a critical step in conveying the need for resubmission, providing feedback

on issues detected.

Assuming all administrative checks are passed, a contract is issued to the Artist, and the process hinges on the timely signing and submission of this contract within seven days. Failure to meet this deadline results in a notification and process termination, while compliance signifies the successful completion of the song submission.

The diagram's structured layout ensures clarity in navigating the sequential stages, highlighting decision points and outcomes. The notification precedes process termination, signifying its role as a preparatory step for the Artist to address issues in subsequent submissions. The terminated instance requires the Artist to initiate a new instance to resubmit their song, with the notification bridging the gap between instances.

This BPMN diagram assumes that communications between the two pools are facilitated through throwing-catching messages. The process termination signifies a complete stop, requiring the Artist to submit a new instance for a restart. The notification plays a crucial role by informing the Artist of issues and serving as a bridge between terminated and new instances, enabling them to address and resubmit their song effectively.

6 | CONCLUSION

In summary, this project successfully addressed the challenging task of developing a robust data structure for an application providing music streaming services. By following the Entity-Relationship (ER) model that was drawn up in the first project delivery, Neo4j's AuraDB was used for the construction of the database.

This DB is capable of storing, retrieving, and representing accurately in a graph form relevant pieces of information in the domain of the relationships between entities, in this case better called nodes. In order to validate the effective functioning of the proposed design, a series of example queries was run (by using Cypher language) and displayed in the document each one with its respective output.

Then, a BPMN diagram was designed by following the second delivery's directives, which was highlighting the processes between artists who want to publish a song or podcast into the 'MusicApp' and the system itself, in which there are checks made by both the system and administrators before confirming the release of the audio content.

To conclude, this project accomplished the primary objective of data design modelling and constructing via a methodical path from design to practical implementation, thus resulting in a coherent and functional music application database. The DB is able to promptly satisfy the requirements of keeping and accessing relevant data whenever it's needed, by focusing more relationships thanks to the peculiar functionalities of Neo4j.

